MOD -03 -> Advanced Java

LUMINAR
TECHNOLAB

Java Thread

LUMINAR
TECHNOLAB

Multitasking

Multitasking

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU.

Multitasking can be achieved by two ways:

1. Process-based Multitasking(Multiprocessing)
2. Thread-based Multitasking(Multithreading)

1)Process-based Multitasking(Multiprocessing)

1. Each process have its own address in memory i.e. each process allocates separate memory area.
2. Process is heavyweight.
3. Cost of communication between the process is high.
4. Switching from one process to another require some time for saving and loading registers, memory maps, updating lists etc.

2) Thread-based Multitasking(Multithreading)

1. Threads share the same address space.
2. Thread is lightweight.
3. Cost of communication between the thread is low.

Java Multithreading

1. Multithreading in java is a process of executing multiple threads simultaneously.
2. Thread is basically a lightweight sub-process, a smallest unit of processing.
3. Multiprocessing and multithreading, both are used to achieve multitasking.
4. But we use multithreading than multiprocessing because threads share a common memory area.
5. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.
6. Java Multithreading is mostly used in games, animation etc.

Advantages of Java Multithreading

1. It doesn't block the user because threads are independent, and you can perform multiple operations at same time.
2. You can perform many operations together, so it saves time.
3. Threads are independent so it doesn't affect other threads if exception occur in a single thread.

Synchronizing Threads

Synchronization in java is the capability to control the access of multiple threads to any shared resource.

Java Synchronization is better option where we want to allow only one thread to access the shared resource.

What is synchronized in Java?

1. synchronized is not a class
2. synchronized is not an interface
3. synchronized is a keyword in Java. It is used to ensure thread safety by allowing only one thread at a time to access a block of code or method.

Types of Synchronization

There are two types of synchronization

1. Process Synchronization
2. Thread Synchronization

Thread Synchronization

There are two types of thread synchronization mutual exclusive and inter-thread communication

1) Mutual Exclusive
    1. Synchronized method.
    2. Synchronized block.
    3. Static synchronization.
2) Cooperation (Inter-thread communication in java)

Mutual Exclusive

Mutual Exclusive helps keep threads from interfering with one another while sharing data. This can be done by three ways in java:

1. by synchronized method
2. by synchronized block
3. by static synchronization

Concept of Lock in Java

- Synchronization is built around an internal entity known as the lock / monitor.
- Every object has a lock associated with it.
- By convention, a thread that needs consistent access to an object's fields has to acquire the object's lock before accessing them, and then release the lock when it's done with them.

- From Java 5 the package java.util.concurrent.locks contains several lock implementations.

Understanding the problem without Synchronization

In this example, there is no synchronization, so output is inconsistent.

```
class Table{

void printTable(int n){//method not synchronized
   for(int i=1;i<=5;i++){
     System.out.println(n*i);
     try{
      Thread.sleep(400);
     }catch(Exception e){
     e.printStackTrace();;
     }
   }
 }

}
```

```java
class MyThreadOne extends Thread{
    Table t;

    MyThreadOne(Table t){
        this.t=t;
    }

    public void run(){
        t.printTable(5);
    }

}
class MyThreadTwo extends Thread{
    Table t;

    MyThreadTwo(Table t){
        this.t=t;
    }

    public void run(){
        t.printTable(100);
    }
}
```

```java
public class TestSynchronization{
    public static void main(String args[]){

        Table obj = new Table();

        MyThreadOne t1=new MyThreadOne(obj);
        MyThreadTwo t2=new MyThreadTwo(obj);

        t1.start();
        t2.start();
    }
}
```

Java Synchronized Method

- If you declare any method as synchronized, it is known as synchronized method.
- Synchronized method is used to lock an object for any shared resource.
- When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

```
Synchronized void printTable(int n){        //synchronized method

    for(int i=1;i<=5;i++){
        System.out.println(n*i);
        try{
            Thread.sleep(400);
        }catch(Exception e){
            e.printStackTrace();;
        }
    }
}
```

Java Synchronized Block

Synchronized block can be used to perform synchronization on any specific resource of the method.

Suppose you have 50 lines of code in your method, but you want to synchronize only 5 lines, you can use synchronized block.

If you put all the codes of the method in the synchronized block, it will work same as the synchronized method.

Points to remember for Synchronized block

- ✓ Synchronized block is used to lock an object for any shared resource.
- ✓ Scope of synchronized block is smaller than the method.

```
void printTable(int n){
   synchronized(this){//synchronized block
     for(int i=1;i<=5;i++){
      System.out.println(n*i);
      try{
       Thread.sleep(400);
      }catch(Exception e){
      e.printStackTrace();;
      }
     }
   }
 }
```

Static Synchronization

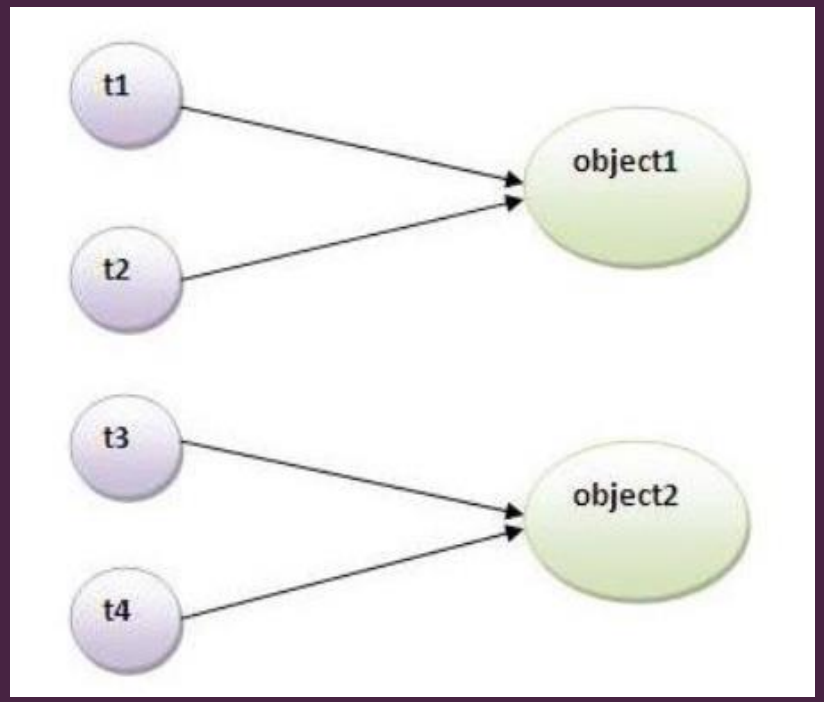
If you make any static method as synchronized, the lock will be on the class not on object.

Problem Without Static Synchronization

Suppose there are two objects of a shared class(e.g. Table) named object1 and object2.

In case of synchronized method and synchronized block there cannot be interference between t1 and t2 or t3 and t4 because t1 and t2 both refers to a common object that have a single lock.

But there can be interference between t1 and t3 or t2 and t4 because t1 acquires another lock and t3 acquires another lock.We want no interference between t1 and t3 or t2 and t4. Static synchronization solves this problem.



In this example we are applying synchronized keyword on the static method to perform static synchronization.

```
package threaddemos;


class Tablez{
 synchronized static void printTable(int n){
   for(int i=1;i<=10;i++){
     System.out.println(n*i);
     try{
       Thread.sleep(400);
     }catch(Exception e){}
   }
 }
}
```

```
class MyThreadOnez extends Thread{
    public void run(){
        Tablez.printTable(1);
    }
}

class MyThreadTwoz extends Thread{
    public void run(){
        Tablez.printTable(10);
    }
}
class MyThreadThird extends Thread{
    public void run(){
        Tablez.printTable(100);
    }
}
class MyThreadFourth extends Thread{
    public void run(){
        Tablez.printTable(1000);
    }
}
```

```
public class TestStaticSynchronization {

    public static void main(String t[]){
        MyThreadOnez t1=new MyThreadOnez();
        MyThreadTwoz t2=new MyThreadTwoz();
        MyThreadThird t3=new MyThreadThird();
        MyThreadFourth t4=new MyThreadFourth();
        t1.start();
        t2.start();
        t3.start();
        t4.start();
    }
}
```

Thank you ☺ Happy learning ☺